SwiftDrop
Backend Task

Q.E.D. — Web Dev Bootcamp |Task 1

The Situation

SwiftDrop launched 2 weeks ago. Customers are signing up — but the backend is broken.

- Can’t search or filter packages
- Wrong ID crashes the server

- Cancelled a **delivered** package — $200 lost
- Double-click creates duplicate shipments

The previous developer quit.

You are the replacement. 4 tickets are waiting.

What is SwiftDrop?

A package delivery platform — think of it as a simplified Aramex or Bosta backend.

Packages

Each package has a sender, recipient, city, weight, priority, and a tracking code.

Couriers

Drivers who pick up and deliver packages. Each has a name, phone, vehicle, and availability status.

Statuses

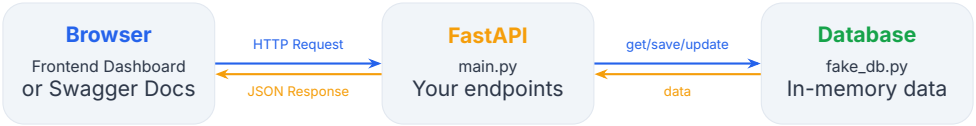
Every package moves through a lifecycle: pending → assigned → picked up → delivered.

The API connects the frontend dashboard to the database.

Customers create packages, track status, filter by city — all through your endpoints.

How It Works

The frontend sends HTTP requests to your API. Your API reads/writes the database and responds.



What the frontend does:

- Sends `GET`, `POST`, `PATCH`
- Displays data in cards & tables
- Shows error messages to users

What your API does:

- Validates input (Pydantic + rules)
- Reads/writes the database
- Returns JSON with status codes

Project Structure

swiftdrop-task/

- main.py
- utils.py
- schemas.py
- fake_db.py
- requirements.txt
- frontend/
  - index.html

File	Action
`main.py`	**Your code** — Tickets 1–4
`utils.py`	**Your code** — Bonus
`schemas.py`	Read only — validation
`fake_db.py`	Read only — database helpers
`index.html`	Open in browser to test

```
pip install -r requirements.txt
uvicorn main:app --reload
```

The Database — fake_db.py

Each package stored in memory looks like this:

```
{
    "id": 1,
    "tracking_code": "SD-00001",
    "sender_name": "Ahmed Hassan",
    "recipient_name": "Sara Mohamed",
    "city": "Cairo",
    "weight_kg": 2.5,
    "fragile": True,
    "priority": "express",       # standard | express | same_day
    "status": "pending",         # pending | assigned | picked_up | ...
    "assigned_courier_id": None,
}
```

Pro Tip: In a real app, this would be a Postgres or MongoDB database. Here, it's just a Python dictionary in memory!

Your Toolbox

Use these helpers in your code — don't access the data directly.

Database Helpers

`get_all_packages()`
→ list of all packages

`get_package_by_id(5)`
→ one package or None

`save_package({...})`
→ saves with auto-generated ID

`update_package(5, {...})`
→ updates specific fields

`delete_package(5)`
→ True or False

Status Rules

`VALID_TRANSITIONS` dict

`"pending"` → assigned, cancelled
`"assigned"` → picked_up, cancelled
`"picked_up"` → in_transit, cancelled
`"in_transit"` → delivered, cancelled
`"delivered"` → {} (final)
`"cancelled"` → {} (final)

Already Built — Study These Patterns

The previous developer finished some endpoints. Study the code to learn the patterns.

Fee Calculator — GET /packages/{id}/fee

```
# Weight-tier pricing
if weight <= 5:    base = 30
elif weight <= 15: base = 60
elif weight <= 30: base = 100
else:              base = 180

# Priority multiplier
multipliers = {"standard": 1.0, "express": 1.5, "same_day": 2.0}

# Final price
total = (base * multipliers[priority]) + (25 if fragile else 0)
```

Base tiers × multipliers + surcharges

Ticket #1 — Get Package Details

BUG Server crashes on wrong ID

Scenario:

Customer enters a wrong package ID. Server returns `500` instead of a helpful error message.

What to do:

1. Look up with `get_package_by_id()`
2. Not found? → `HTTPException(404)`
3. Found? → return the package

Path Parameters | Error Handling

Test Cases:

Request
`GET /packages/1`

Expected — 200 OK
```
{"id": 1, "city": "Cairo", ...}
```

Request
`GET /packages/9999`

Expected — 404
```
{"detail": "Package not found"}
```

Ticket #2 — List & Filter Packages

FEATURE No search or filter

Scenario:

GET /packages returns everything in one list. Frontend sends query params but API ignores them.

What to do:

1. Get all from `get_all_packages()`
2. Filter by `city`, `status`, `priority`
3. Paginate: `[offset : offset+limit]`
4. Return `{"total": N, "packages": [...]}`

Query Parameters CRUD — Read

Test Cases:

All packages
`GET /packages`
→ {"total": 15, "packages": [...]}

Filter by city
`GET /packages?city=cairo`
→ only Cairo packages

Paginate
`GET /packages?limit=3&offset=6`
→ 3 items, skip first 6

Ticket #3 — Create Package

FEATURE No business validation

Scenario:

Pydantic rejects bad types automatically. But no business rules exist. A customer sent a package to themselves.

Business rules to add:

- sender = recipient → **400**
- same_day + weight > 10kg → **400**
- fragile + weight > 30kg → **400**

Then `save_package()` → **201**

Pydantic | Status 201 | Validation

Test Cases:

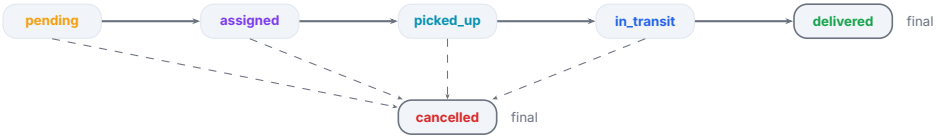
Valid request
`POST /packages`
→ 201 Created + tracking code

Sender = Recipient
`POST /packages`
→ 400 “Cannot be the same”

Bad type (Pydantic auto)
`weight: "heavy"`
→ 422 Validation Error

Ticket #4 — Update Delivery Status

BUG Delivered package cancelled — $200 lost



What to do:

1. Get package (404 if missing)
2. Check `VALID_TRANSITIONS`
3. Invalid? → **400**
4. Valid? → `update_package()`

CRUD — Update State Machine

Valid: pending → cancelled
`PATCH /packages/5/status` → 200

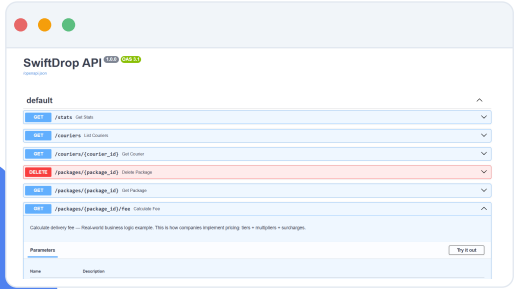
Invalid: delivered is final
`PATCH /packages/3/status` → 400

Testing Your Work

You have two distinct ways to verify your backend. Test early, test often!

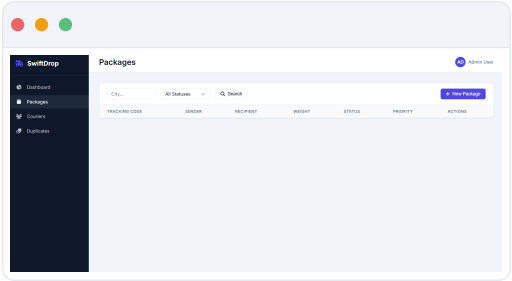
METHOD 1 Swagger Docs

FastAPI generates a live testing environment. Go to `localhost:8000/docs`, click **Try it out**, and execute requests.



METHOD 2 Frontend Dashboard

Open `frontend/index.html` in your browser. If your API is correct, the UI buttons here will work seamlessly.



Bonus #1 — Duplicate Detection

BONUS Double-click “Send” ships twice — we lose money

Naive — O(n^2)

Compare *every* package against *every* other using nested loops.

2,000 packages = 4,000,000 comparisons

Server timeout

⇒

Optimized — O(n)

Group by `(sender, recipient, weight)` into a dict. Compare only within each group.

2,000 packages ≈ 2,000 ops

30x faster

Write `find_duplicates_optimized()` in `utils.py`

Hint: use a dict with tuple keys to group packages, then compare timestamps within each group

Bonus #2 — Maximum Deliveries

BONUS 6 requests overlap. Maximize deliveries.

Greedy: A, C, D, F = 4 deliveries

Naive: A, D, F = 3 deliveries

The greedy algorithm: Sort by end time (earliest first). Pick first. For each next — if it starts after the last ended, take it. Skip overlaps.

Write `select_max_deliveries_optimized()` in `utils.py`